

Top 50 Database Interview Questions

SQL, NOSQL, MONGODB, MONGOOSE, & POSTGRESQL • JUNIOR DEVELOPER LEVEL

Section 1: General Database Concepts & SQL vs. NoSQL

1. What is the fundamental difference between SQL and NoSQL databases?

SQL databases are relational, table-based, and use a strict, predefined schema. NoSQL databases are non-relational, document- or key-value-based, and feature a dynamic schema for unstructured data.

2. When would you choose a SQL database over a NoSQL database for a project?

Choose SQL when data integrity, complex multi-table joins, and strict ACID compliance are required (e.g., financial apps). Choose NoSQL when your data structure changes rapidly or you need massive horizontal scaling for high-speed reads/writes.

3. What does ACID compliance mean in relational databases?

ACID stands for Atomicity (all or nothing transactions), Consistency (valid schema state), Isolation (independent concurrent execution), and Durability (permanently saved data). It guarantees that database transactions are processed reliably.

4. What is the CAP Theorem, and how does it apply to NoSQL databases?

The CAP Theorem states that a distributed system can only guarantee two out of three properties at once: Consistency, Availability, and Partition Tolerance. NoSQL systems must trade off absolute availability to ensure consistency, or vice versa, during network splits.

5. What is the difference between vertical scaling and horizontal scaling?

Vertical scaling means adding more power (CPU, RAM) to a single existing server, which is typical for SQL. Horizontal scaling means adding more machines to your network to share the load, which NoSQL handles natively.

6. What is a database schema, and what does it mean when we say NoSQL is "schemaless"?

A schema is the blueprint defining how data tables, columns, and types are structured. NoSQL being "schemaless" means documents in the same collection do not have to share the exact same fields or structure.

7. What is Data Normalization, and why do we use it in relational databases?

Normalization is the process of organizing table structures to minimize data redundancy and dependencies. It prevents data anomalies during inserts, updates, and deletes by splitting data into linked tables.

8. What are the downsides of over-normalizing a database?

Over-normalization splits data across too many tables, requiring the system to perform complex `JOIN` operations. This degrades read query performance and makes SQL queries highly complex to write.

9. What is Denormalization, and why is it common in NoSQL?

Denormalization is the intentional strategy of adding redundant or grouped data to optimize read performance. It is common in NoSQL because fetching a single, self-contained document is much faster than performing distributed joins.

10. What is a Database Index, and how does it improve query performance? What is the catch?

An index is a pointer structure that allows the database to find rows instantly without scanning the entire table. The trade-off is that it slows down `INSERT`, `UPDATE`, and `DELETE` actions and uses extra disk space.

Section 2: Relational Databases & PostgreSQL

11. What is the difference between DBMS and RDBMS?

A DBMS is any software used to store and manage data files. An RDBMS is a specific type of DBMS that organizes data strictly into relational tables and enforces ACID properties (like PostgreSQL).

12. What are the differences between Primary Keys, Foreign Keys, and Unique Keys?

A Primary Key uniquely identifies each row in a table and cannot be `NULL`. A Foreign Key links to a primary key in another table to enforce a relationship, while a Unique Key ensures all values in a column are distinct but can accept `NULL`.

13. Explain the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

`INNER JOIN` returns records with matching values in both tables. `LEFT JOIN` returns all records from the left table plus matches from the right; `RIGHT JOIN` does the opposite. `FULL OUTER JOIN` returns all records when there is a match in either table.

14. What is the purpose of the GROUP BY clause, and how does it differ from ORDER BY?

GROUP BY collapses rows with identical data into summary groups, often used with aggregate functions like COUNT() or SUM(). ORDER BY simply sorts the final result set in ascending or descending order.

15. What is the difference between the WHERE clause and the HAVING clause?

WHERE filters individual data rows *before* any groupings or aggregations take place. HAVING filters groups *after* the GROUP BY clause has been executed and works with aggregate functions.

16. Explain the difference between DROP, TRUNCATE, and DELETE commands in SQL.

DELETE removes specific rows based on a condition and can be rolled back. TRUNCATE rapidly removes all rows from a table but keeps the structure, while DROP completely deletes the entire table structure and data from the database.

17. What are DDL and DML? Give examples of each.

DDL (Data Definition Language) defines or alters the database structure using commands like CREATE and DROP. DML (Data Manipulation Language) manages the actual data within those structures using SELECT, INSERT, and UPDATE.

18. What is a Subquery (or nested query), and when would you use one?

A subquery is a query nested inside another SQL statement's WHERE, FROM, or SELECT clause. You use it when your data filtering criteria depends on the dynamic output of a separate query.

19. What is a View in PostgreSQL, and how does it differ from a physical table?

A View is a virtual table representing the saved result of a pre-written SELECT query. It does not store physical data itself; instead, it dynamically runs its underlying query whenever you fetch data from it.

20. What is a Constraint in PostgreSQL? Name three common types.

A constraint is a structural rule applied to a table column to enforce data integrity. Common types include NOT NULL (prevents empty values), UNIQUE (forces distinct values), and CHECK (validates a custom boolean condition).

21. What is an Execution Plan (EXPLAIN in PostgreSQL), and how do you use it to debug a slow query?

Pre-pending EXPLAIN ANALYZE to your query makes PostgreSQL return an execution plan instead of data results. It reveals whether the database is using slow table scans or fast index scans, pinpointing performance bottlenecks.

22. How does PostgreSQL handle auto-incrementing fields?

PostgreSQL uses internal sequence generators. In modern SQL syntax, this is implemented by applying the identity clause `GENERATED ALWAYS AS IDENTITY` to an integer primary key column.

23. What is a Transaction, and how do you use COMMIT and ROLLBACK in SQL?

A transaction groups multiple SQL queries into a single unit of work. `COMMIT` permanently saves all modifications made during the transaction, while `ROLLBACK` cancels all changes if an error occurs mid-execution.

24. What are Aggregate Functions? Provide three examples.

Aggregate functions perform mathematical calculations on a set of column values and return a single scalar value. Examples include `COUNT()` (total rows), `SUM()` (total addition), and `AVG()` (mean average).

25. How do you find duplicate rows in a SQL table using a query?

You can group the table by the target columns and filter those groups using a `HAVING` clause where the count of occurrences is greater than one.

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

Section 3: NoSQL & MongoDB

26. What is MongoDB, and how does it store data internally?

MongoDB is a document-oriented NoSQL database. While developers query it using human-readable JSON formats, it stores data internally as BSON (Binary JSON) to accommodate advanced data types and enable faster machine parsing.

27. In MongoDB, what are Collections and Documents? What are their SQL equivalents?

A Document is a single data record consisting of key-value pairs, equivalent to a SQL Row. A Collection is a grouping of these documents, which is equivalent to a SQL Table.

28. What is the purpose of the `_id` field in a MongoDB document? How is it generated?

The `_id` field acts as the unique primary key for every document. If you do not provide one, MongoDB automatically generates a unique 12-byte hex string called an `ObjectId` containing a timestamp and machine identifier.

29. How do you perform basic CRUD operations in MongoDB?

CRUD operations are executed via target collection methods: `db.collection.insertOne()` to create, `.find()` to read, `.updateOne()` with a `$set` operator to modify, and `.deleteOne()` to remove data.

30. What is the difference between `db.collection.find()` and `db.collection.findOne()`?

`find()` returns a cursor pointer representing an array of all matching documents that you can iterate through. `findOne()` scans until it finds the first matching document and immediately returns that single object.

31. How does MongoDB handle relationships between data? Explain Embedding vs. Referencing.

Embedding stores related child data directly inside a parent document as an embedded object or array. Referencing stores an `ObjectId` pointer that links to a document kept in an entirely separate collection.

32. When would you prefer embedding a document over referencing another document in MongoDB?

Prefer embedding when data is clean, bounded, tightly related, and frequently read together. Avoid it if the child array grows infinitely, as MongoDB limits an individual document's maximum size to 16MB.

33. What is the Aggregation Framework in MongoDB, and what is a pipeline?

The Aggregation Framework is an advanced data processing tool used to compute analytics. A pipeline passes documents through sequential processing stages that filter, group, and transform records on the server.

34. Name three common stages in a MongoDB aggregation pipeline and what they do.

`$match` filters documents to pass only specific matching criteria (like a SQL `WHERE`). `$group` bundles input documents together by a specified key to compute metrics, and `$sort` reorders the final layout.

35. What is Sharding in MongoDB, and how does it help with high volumes of data?

Sharding is MongoDB's mechanism for horizontal scaling. It breaks up large datasets and distributes the partitions across multiple physical servers (shards) to increase storage capacity and write throughput.

36. What is a Replica Set in MongoDB, and why is it important?

A Replica Set is a cluster of MongoDB nodes that sync identical data copies. It features one Primary node for writes and multiple Secondary nodes; if the primary crashes, automated failover crowns a secondary node to prevent downtime.

37. How do you create an index in MongoDB, and how do you verify if a query is using it?

You create an index using `db.collection.createIndex({ fieldName: 1 })`. To verify if a query is actively utilizing that index, append `.explain("executionStats")` to your target query string.

38. What is a TTL (Time-To-Live) Index in MongoDB, and what is a typical use case for it?

A TTL index is a special single-field index that automatically deletes documents from a collection after a specified duration. It is commonly used to clean up temporary sessions, OTP tokens, or system logs.

Section 4: Mongoose (ODM for MongoDB/Node.js)

39. What is Mongoose, and why do we use it if MongoDB is already schemaless?

Mongoose is an Object Data Modeling (ODM) library for Node.js. We use it to enforce structure, write strict data validation rules, and handle type casting on the application layer before data reaches a schemaless MongoDB database.

40. What is the difference between a Mongoose Schema and a Mongoose Model?

A Schema is the architectural blueprint that defines properties, types, and validations for a collection. A Model is a compiled constructor class built from the schema that provides the CRUD methods (e.g., `User.find()`).

41. How do you define a validation rule in a Mongoose schema?

You apply a configuration object to a schema property, defining restrictions like `required: true`, `unique: true`, or setting built-in min/max parameters alongside custom validation error messages.

42. What are Mongoose Middlewares (Hooks), and what is the difference between a pre and post hook?

Middlewares are functions that intercept actions during the model lifecycle. `pre` hooks execute *before* an event (e.g., hashing a password before saving), while `post` hooks trigger *after* an event completes (e.g., sending a welcome email).

43. Explain how `.populate()` works in Mongoose. What SQL concept does it mimic?

`.populate()` lets you reference documents across collections. Mongoose automatically reads stored reference IDs within a document and queries the target collection to inject the full objects, mimicking a SQL `JOIN`.

44. What is the difference between `Model.create()` and `Model.save()` in Mongoose?

`Model.create()` is a convenience method that instantiates and saves a document to the database in a single step. `Model.save()` is an instance method called on a manually pre-constructed document object, useful for modifying fields before saving.

45. How do you handle database connection errors or disconnections in a Node.js/Mongoose application?

You register event listeners on the active `mongoose.connection` object, listening for specific event strings like `'error'` or `'disconnected'` to trigger error handling or reconnection logic.

46. What are Virtuals in Mongoose? Provide a practical example.

Virtuals are document properties that you can get and set but are not saved to the underlying MongoDB database. A common example is combining `firstName` and `lastName` fields to return a transient `fullName` property.

Section 5: Practical & Scenario-Based Questions

47. If an application query takes 5 seconds to load, how do you troubleshoot and optimize it?

First, use query explain tools (`EXPLAIN ANALYZE` or `.explain()`) to check for full table scans. If confirmed, build a targeting database index on the filtered fields, and restrict data retrieval to only necessary columns.

48. You need to store user blog posts with a fluid number of tags and comments. Would you choose PostgreSQL or MongoDB?

Choose MongoDB because its document model fits flexible, hierarchical structures naturally. It allows you to embed or reference unpredictable arrays of tags and comments together in a single fast read operation without heavy multi-table SQL joins.

49. If a junior developer pushes database credentials to a public GitHub repository, what steps should be taken?

Immediately revoke and rotate the leaked connection credentials on the database provider host. Update the application to reference local `.env` variables, and use a tool like `git-filter-repo` to permanently erase the compromised history from Git tracking.

50. What is a SQL Injection attack, and how do ORMs or parameterized queries protect against it?

A SQL Injection happens when untrusted user input is directly concatenated into a raw SQL query string, letting attackers run unauthorized code. Parameterized queries protect against this by treating input strictly as literal values rather than executable syntax.